

Lec 56 Evolution From LSTMs to Transformers

Lec 56 Evolution From LSTMs to Transformers

Lecture	56 / 63 + intro
Duration	42:27
Captions	English (auto-generated)
Words	5,294
Video	PkK0LcbqPXM
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone, welcome back to the lecture series on fundamentals of generative AI and large language models. In my previous session, we have spoken about the sequential models like RNNs and LSTMs. In today's session, we will talk about evolution from LSTMs to transformers. Okay. So the contents for today's session goes like evolution of sequential models, limitations of RNN and LSTMs, attention mechanism, encoder decoder attention, self attention, multi head attention, masked self attention, introduction to transformers, RNN's, LSTMs versus transformers. Okay. So we will start with the evolution of sequential models. So in my previous lecture we have seen the we have studied about RNNs and LSTMs which are very

[00:52] well-known sequential models. Okay. Now we will see after this one how it evolved towards transformers. Okay. So in 2013 RNN came into picture and then LSTMs. LSTMs were introduced in 1997 but became widely adopted after 2014. Okay. And then in 2014 itself attention mechanism which is one of the very very significant um uh part or one of the very very significantly contributing factor to transformers is introduced in 2014. So we will look into what is this attention mechanism in this today's uh session.

[01:42] And then in 2017 transformers came into picture that's the you know the entire landscape of NLP has changed in after 2017. So with the with the introduction of transformers in 2017 we came to bird GPT5 lama so on and so forth. Okay. So we have so many models now in the transformer architecture. Okay. So that is the you know evolution over a period of time from RNN to transformers and we will see how the limitations that were faced in LN uh RNN and LSTMs they are overcome in transformers. Okay. So let us have a quick look into what are the limitations of RNN and LSTMs.

[02:31] Suppose just take this example. The book that I brought yesterday from the new bookstore is amazing. Okay. So there is a sequential dependency which what I mean by this one is the book that I brought yesterday from the new bookstore is amazing. This is basically is talking about the book is or related to the book. Okay. And there is a sequence of hidden states that the RNN uh you know produces to remember this information or to capture this dependency. So there is a sequential dependency. Okay. So so many hidden states will be generated to find out or to memorize that this is actually talking about this book. Okay. So recurrent models generate a sequence of hidden states h_t and h_t keeps the memory

[03:34] of the previous words where h_t as we have seen in our last lecture also is equal to $\tanh(v \cdot h_{t-1} + u \cdot x_t + b_h)$. h_{t-1} is the hidden state from the $t-1$ time step. x_t is the input at the t th time step. b_h is the bias term associated with the you know hidden state and V and U are the weight matrices. Okay. So there is a sequential dependency for capturing these you know uh relationship between words in a sentence.

[04:14] Now another example think of this um uh sentence the quick brown fox. Okay to understand the word brown the model first understand or first process the word quick and this quick depends on that there is a relationship between these two. So there is a sequence of steps which needs to be followed and each and every word will have a hidden state. So there will be so many hidden states that will be sequence of hidden states will be there.

[04:48] So this have a memory constraint for long sequences. What happens is the computers they should hold a sequence of you know hidden states so that they can remember the information that was there at the beginning of the sentence and keep it till the end of the sentence. Okay. So this have a very very strict memory constraint and together with the memory problem there is another problem that is associated the information from one word in the initial part of the sentence. If it is having a relationship with one word towards the end of the sentence, the information between these distant words must travel through big $O(N)$ sequential steps means that which to make it more simple if you have thousand

[05:42] words in the sequence it should involve thousand operations. So this is a

problem or challenge of long-term dependency. If you have thousand words, the computer needs to go, you know, process thousand steps to keep the memory of the word which is um uh which is previous to these of a word in thousand steps previous to this word. Okay. So this becomes computationally expensive. Okay. So when we speak about you may ask how would this thousand words how in which kind of uh scenarios these thousand words come into picture scenarios like if you have a paragraph and there is something written in the first line of the paragraph and you know maybe middle of the paragraph. So that kind of long range dependencies become

[06:36] challenging for RNN. Okay. RNNs and LSTMs. Then came into picture because of these uh complications or these challenges there was a new concept that was introduced which is called as the attention mechanism. Now what is this attention idea? Scientists started thinking that when predicting a word, do we really need to know all the previous hidden states equally? Is it important or is it relevant that we require all the hidden states? We need to remember them equally. The answer is no.

[07:22] Because the model should because what happens is maybe just the you know preceding word or two preceding words may not have any linkage with that one or very limited amount of relationship with the word in question. So the model should assign greater importance to most relevant words in the sequence. Okay. So just have a look in the example, it will become much more clearer. The animal didn't cross the street because it was too tired. This is the sentence now because it is too tired. This it is related to animal.

[08:09] Okay. So do I really need to remember this because street da didn't cross all these things equally? Yes we will remember but do we need to give equal importance to all the words when it comes to it? The word that requires the most attention is animal because the it is actually referring to the animal. Okay. So that is the concept of attention. Now that means the idea is simple. Instead of putting equal weightage to all the hidden states, we will put our importance or give some attention weight or attention score to the relevant words in the sequence.

[09:04] Now let us see how that comes into you know in implementation. So let us have a look how attention mechanism is implemented in encoder decoder architecture. Just have a quick look into the encoder decoder architecture. So if we have an encoder which whose work is mainly to take the input sequence and then that information about the input sequence is compressed is is placed in a compressed form and it is given to the decoder through something called as the thought vector. Okay. So this is encoder decod decoder a simple way of understanding how it works. Okay. Now we will see how attention is incorporated in this encoder decoder architecture.

[09:57] The decoder what is the function of the decoder? Decod decoder will predict the output sentences a sentence word by word. Okay. So we will have a look how this entire thing works. Okay. Suppose these are my RNN or LSTM blocks can be

birectional and I have the input sequence. I love machine learning. So this is my input sequence and then it goes to the hidden states $H_1 H_2 H_3 H_T$ okay capital T and then there is something like α_{11} .

[10:37] I'm putting α_{11} here. What is this α ? α denotes the amount of attention to be paid to each hidden state. Okay. So the attention that needs to be put to the hidden state H_1 is given by α_1 . Attention that needs to be put at H_2 is α_2 one. Attention to be put in the hidden state H_3 is given by α_3 . Attention to be put in the hidden state H_T is given by α_T capital T. And they all these go and create one context vector C_1 . Now what is this C_1 ?

[11:30] C_1 is equal to summation over $J = 1$ to T $\alpha_1 J H_J$. Okay. What is this one? You may question what is this one? J . You understand? J is the running you know this uh 1 1 2 1 3 1 4. Okay. That running index. What is this one coming from? 1 1 2 1 3 1 4. What is this one? Okay, this one actually tells you that this context C_1 will be used to predict the first word in the decoder. Okay. So that is where your one comes into picture. So α_1 one. So if we see α_{ij} is the attention weight assigned to the j input word through the hidden state while generating the i th output word.

[12:34] Okay. So that is your α_{ij} . Okay. I hope that is clear. So here there are four words. I love machine learning. So when the decoder predicts the output, it will predict four words. Correct? Suppose I am talking about a language translation from English to German. So my input sequence here is in English. I love machine learning. I want my encoder decoder model with a tension mechanism to give me the equivalent sentence in German.

[13:19] So the entire context here is we are speaking about a language translation where the input language is English which is given in your encoder and you expect the model to give you the German translation uh translation of that uh sentence English sentence which will be given as the output of the decoder. and while it is giving the output it will give one word at a time. Okay. So that is the context. So we have seen how C_1 is calculated. Similarly this is α_{21} α_{22} α_{23} and α_{24} which goes for this is α_{23} this is α_{24} . Okay. This goes to create context C_2 . Okay. And then there is α_{31} α_{32} α_{33} α_{34} which goes to create context C_3 and another set which goes α_{41} α_{42} α_{43} α_{44}

[14:34] they go to create context C_4 . Okay. Now these context vectors $C_1 C_2 C_3 C_4$ if I want to generalize it I can write $C_T = \sum_{J=1}^T \alpha_{tj} H_J$ that's a generalized formula okay now we will see this is what is my encoder part which have taken this is my input sequence and I have created four context vectors which will be used while the decoder wants to predict all the four German translations of these four English words. So we will see how that happens now.

[15:28] So suppose this is my decoder's block. So it takes the initial states and then a beginning of sentence token and then it takes this context C_1 and it predicts the

word it. Okay, which is the German of I. And then what happens is this it comes as the input for the next block. Okay, next block of RN inter LSTM whatever is there and this C2 gives goes as the input to this block as well. So there are two inputs to this block. What are they? One is whatever is predicted here that come that has already come here and the context C2 also have come here.

[16:25] Okay. and then we get the predicted word like. Okay. Similarly, when we are trying to predict the third word, then this whatever is predicted in the second word comes as the input to the third block and together with the context C3 and you get the third word machineliness. Okay. And then similarly this comes here as an input to the fourth block which gives together with context C4 also comes as the input here and you got learning.

[17:12] Okay, which is the German of learning. This thing is repeated here. This one comes as the input for your last block and then it gives the end of statement token. Okay. So this is the way the attention mechanism is incorporated inside an encoder decoder architecture. Okay, this is the mechanism by which so the attention mechanism is actually working between the encoder's input sequence and the out decoder's output sequence.

[17:53] So this is about attention in encoder decoder architecture. So this is basically your decoder. So that is what has been shown here. Now let us move from attention to self attention or see let us understand attention is the bigger umbrella word inside that one. What is this self attention and is there any other kind of attention that is there? So in the encoder decoder attention the attention mechanism as I was telling is between the input words which comes from the encoder and the output words which are predicted by the decoder. Okay. So that is the attention in encoder decoder architecture.

[18:39] Now there is the term self attention. What is the idea of this self attention? In self attention, every word learns from every other word. I love machine learning. Okay. So, I will learn from the second word love. Love will learn from the word machine. Machine will learn from the word learning. Learning will learn from all the other words that are there. So all combinations that means every word learns from every other word for that one each token is projected into three vectors. So I love machine learning that all these words will have a query vector, a key vector and a value vector.

[19:35] These are very very important things query key and value. We keep on talking about them. We will be using more. Okay. So, query vector, key vector and a value vector. Okay. Now, example I love generative AI as I was telling tokens are I love generative AI and you have each of these tokens will have embedding vectors and from there you will have you know the query key and value. Now let us take an example. Suppose the tokens are I.

[20:17] This I is actually having the embedding vector look which looks like this. Okay. So it is a vector which is the mathematical expression of the word I. Similarly, other words also will have and we are writing it as X_1 . Similarly, love is having an word embedding vector which is $\begin{bmatrix} 9 \\ 3 \end{bmatrix}$ some numbers and then that is called as the X_2 . Then generative is another vector like looks like this. Similar these are taken these numbers are just for example. Okay.

[20:56] And this is called as x_3 and then a_i this is another vector. These are all the embedding vectors and x_1, x_2, x_3, x_4 . Okay. So this is the word embedding vectors for the tokens. I love generative AI. Okay. Each word will have a query vector, key vector and a value vector. Each of these words or tokens will have a query key and value vector. Okay. The query vector Q is given by XW_Q . What is this W_Q ? It is the weight matrices matrix associated with the query vector. Okay.

[21:39] K is X into W_K . W_K is the weight matrix associated with the key vector and V is the X into W_V weight matrix W_V is equal to the weight matrix associated with the value vector. Okay. So we get the query key and the value vector by multiplying the weight matrices with the X . Okay. From here after we get the query key and value vector we get the tension score which is given by $\text{soft max } q \cdot k^T / \sqrt{d_k}$ d_k is the dimension of the k vector into the value vector. So this is the equation very very important equation for attention.

[22:27] Okay. Now we will look into how this attention Q, K and V is it enough for picking up all the relationships that are there between the various kind various words that are present in the sentence. Just let's have a look into that one. Okay, we are coming to a new concept called as multi head attention. So what is this multi-head attention? Multi-head attention suppose just take this example. The scientist published the paper because she completed the experiment successfully.

[23:12] This is the sentence. Now there are several relationships. See the paper paper has a relationship with published. Experiment has a relationship with completed. Scientist has a relationship with the word completed because the scientist completed. Scientist has a relationship with see she. This is also there. Okay. So there are multiple relationships which are present in the work. The question that came or to the mind of the scientists are is self attention will it be able to capture all these kind of various relationships that are present in a sentence. There will be various kind of linguistic relationships which is possible.

[24:12] Will self attention be able to capture all of them together? That was the question. So a single attention head can only learn one dominant relationship at one time. Okay. So the core idea of multi-head attention is different attention heads will be there which will learn different linguistic relationships simultaneously. Suppose there are four or three attention heads. If head one is learning the grammar, head two might be learning the um semantic relationship. Head three may be dealing with the

syntactic relationship. So there are various kind of relationships okay in a linguistic uh domain.

[25:07] So if we have more number of heads attention heads they can capture the various kinds of relationships. Okay. So that is the main idea of multi head attention. So let's look into this picture. So this picture have couple of layers. So we will see what are they. So we know currently query, key and value these are the three very important parameters when we are talking about attention. So what is this query? Query is basically what I'm looking for that is the question.

[25:47] Key is what information do I contain and value is what information should I pass. So these are the very fundamental things in attention and these three actually go as the input to your multi head attention. So you have the query key and value vectors. Okay. Then they are going to linear layers. Okay. These linear layers what is their work? These linear layers they use learnable weight matrices different learnable weight matrices to create different views of the same sentence.

[26:27] What do I mean by that? Sometimes it will you know from various perspectives it will see and see what are the relationships between all the words that are there. So are we taking up all the relationships that are present there? So that that from that intention they take different views of the same sentence and then there is a layer called projection layer which creates the query key and value representation for one attention head.

[27:00] Now this if you see here there are multiple. So for one attention head one projection will uh create one pair of key query and value representation for one attention head. The second one will create suppose for attention head one. The query key values are are given by the first layer. The second layer will give it for attention head two. The third layer will give it for attention head three. So if you see there are three layers everywhere.

[27:35] Okay. So and this is going to query is going to all the three. This is all the three and all the three layers here. After the query key and value representations are you know created by this projection layer. Then what is the next one? It is basically the attention score calculation or attention weight calculation which is nothing but your that soft max equation. Okay. $Q \cdot K^T$ divided by square root of D_K into V . Okay. So that operation happens. So this is if this is head one, this is two and this is three. Three heads where this that means your self attention or attention is getting calculated in multiple heads and then all these outputs are basically stacked to have one large feature vector. So that is that is the work of

[28:34] the concatenation layer and then finally there is sorry then there is finally another linear layer which combines all these pieces of information into one final representation. That is the way our multi head attention works. Okay. So basically each attention head will have will understand one type of linguistic relationship. Okay.

Someone may understand the grammar part, someone will understand the syntactic part, someone will understand the semantic part like that. Okay. So that is about multi-head attention.

[29:16] Then we will look into what is masked self attention. When we use the word mask that means we are covering something. Now suppose we are talking about a English to French translation. I am a student. It needs to be translated to French juus and edudiant. Okay. Now if I want this English to be translated to French then one word at a time will be predicted correct from the decoder that we have seen. So I am a student.

[29:57] This is the input sequence is given at once and then it goes to the decoder and decoder will generate an output sequence one element at a time. So the output sequence is generated here. Now when the output is predict is predicted suppose it has predicted till this word then the other word should be masked out. We should not be able to see these words. Why? Because while predicting you should not have any information or you will not have any information about the words that are coming after that one. Okay. So we will mask that one. So this is basically the idea of masked self attention. So while predicting a specific word the model should only be aware of the words that came before it.

[30:55] Words which are coming after it the model should not be aware of. So that part should be masked. Okay. So like this J Swiss N. Now it is supposed to predict this. Okay. So when this word is getting predicted this portion is masked and this is the concept of masked self attention. Let us have a look attention matrix without masking and attention matrix with masking how it looks. So if you have the query and key here say I love deep learning the keys are here I love deep learning there is an attention score between all the words I to I to love I to deep I to learning love to I love to love so love to deep and love to learning so this is the attention matrix the numbers I have kept only for example

[31:58] just to make you all understand. Okay. Now if because here there is no masking so every word will have will know each other. Okay. Now think of the situation when there is a masking what is the attention matrix looks like. Suppose you are you know you have only this one I then the other things you are trying to predict love deep learning three words. So you should have only the attention score for I others should be minus infinity. I'll tell why minus infinity.

[32:40] But currently you just understand why all these things are minus infinity. Why only I to attention score or weight? Because when we are trying to predict like I dash dash dash or maybe in translation I have just predicted the French translation of I. The other words are yet to come. I cannot have a attention score here. Similarly suppose I have already done I love and the remaining two predictions are next to word predictions are remaining. For that one I can have the attention score for the

first two words.

[33:24] H but these two should be minus infinity. Similarly if I am doing I love deep dash. So you can have attention score till the third word the fourth should be minus infinity similarly. So that is that is the thing. So when you have gone to the last one then you can have the all the attention scores. Okay. So that is how attention be calculated when there is a masking. Now the question comes why this minus infinity is there?

[34:02] As I was telling the tension matrix without masking every token or we can say it like this every token can attend to the previous tokens to itself and but never to future tokens. That is what is given here in this matrix. It is attending or it is having an attention score with itself. I I it is having with the previous token I love but not with the f future tokens. Sorry, not this one probably. So every token can attend to the previous token itself but never for the future token. Okay. What do I mean by that is suppose you take love it can have a score with itself. It can have a attention score with the previous word which is I but not with the final words.

[35:08] Okay. So this is the concept. Now we will look into why this minus infinity is put. We know attention is calculated using soft max activation function of $\frac{\exp(z_i)}{\sum_{j=1}^k \exp(z_j)}$ square root of d_k into v . This is the equation for attention um calculation. And if we see the soft max activation function is given by sigmoid which is you know $u_m z_i$ it is given by e^{z_i} by summation of $j = 1$ to k e^{z_j} . So if my this term is minus infinity my soft max max will give me or I can say it like this e to the power of minus Sorry.

[36:16] E to the power of minus infinity equal to 0. So when you send it through your soft max whatever is there you get zero. Okay. So that means there is no attention weight that is associated with the words or the future tokens. Okay. So when you are giving the soft max with e to the power of minus infinity it will give you zero. That means there is no attention that has been put with the tokens that are coming in future. The attention weights are attention scores or weights are calculated only for the previous words and the attention to itself. Okay. So this is how the self uh masked attention works. And then there is in 2017 the breakthrough the introduction to transformers came which is a landmark paper

[37:24] which is the attention is all you need by Vaswani et al from Google brain. So it came and it changed the entire picture of NLP. Okay there is no stopping after that. So this paper attention is all you need a very very very very important paper introduce the transformer architecture which works based on two different things one is attention mechanism which we have studied so far and parallel processing. So this attention mechanism can be self attention can be multi-head attention can be encoder decoder attention and it works in parallel processing. Okay. So that's the completely groundbreaking research which came out in 2017 through the paper

attention is all you need by Vaswani et al from Google brain and that

[38:17] completely changed the entire picture of NLP domain. So we will look into uh in my next session we will look into what is this transformers how this transformer architecture it is but the main thing that is the takeaway from today's session is attention mechanism is used and parallel processing is used here. So we'll have a quick look between what is the difference between LSTMs RNN and LSTMs and transformers. So the core idea in transformers basically is instead of reading one word after the other it looks at all the words simultaneously.

[38:55] So we'll have a look into the differences. So RNN and LSTMs reads sequential. Sequentially here in transformers it reads all the tokens together. Hidden states carry memory. In this case attention connects tokens directly difficult to paralyze. And here it is highly highly highly paralyzable. Okay. Slow training for long sequence. Here it is faster training and long distance information may weaken in RNN and LSTMs. Here it is handled very efficiently. So this is just the you know idea of transformers. With this one I come to the end of today's session. In my next session we will look into the transformer architecture.

[39:41] Before I conclude, let us have a look into what all we have studied. So we have seen what are the disadvantages or limitations of RNN and LSTMs particularly in long-term dependencies and then we have talked about the attention mechanism. Why exactly attention mechanism came into picture how we don't want all the uh hidden states to be given equal importance. We will give importance to only the relevant words. That is the idea of attention. Then we have looked into self attention. How attention uh you know uh weights are calculated through the query key and value matrices. Okay. Then we have looked into multi head attention where different attention heads they calculate query key and value for a particular

[40:28] linguistic relationship. And then all the things are concatenated together to give the uh uh you know better picture of the entire uh text. Okay. Better picture means picking up all the linguistic information that is there relationships between the between the words. Then we have looked into what is masked attention and why it is required. We have looked into what why exactly things certain things should be masked. Whatever when we are predicting we can we have to uh we cannot give attention score to the words that are coming. So there can be attention scores that are there with each and every word with itself and the words that have already come. So that is the um uh idea of masked attention masked self attention.

[41:14] Then we have seen that how transformers got introduced in 2017 by the paper attention is all you need by Vaswani et al from Google brain in 2017 Vaswani in 2017 from Google brain and it works on two very important things the attention and parallelization okay so these are the things and finally we have just had a look how LSTMs RNN and LSTMs are different from the transformers Okay. So with this one I

conclude today's session. In my next session we will be talking about the transformer architecture. So till we meet for the next session with the transformer architecture. I urge all of you to go through the paper the attention paper. The attention is all you need. You know go through that paper

[42:04] have a look into it. Go through it so that you become much more conversent with the maths and other things that are there. Okay. So we will look in details into the architecture of transformers in my next session. Till that time happy learning. Thank you all.